

شرح الكود بالتفصيل

شرح مبسط ومفصل للكود المصدري وهياكل البيانات والخوارزميات

CPU Scheduling Simulator Using Appropriate Data Structures

مؤمن عبدالله الشامي	إعداد
د. بلال مرشد	إشراف
C++17	لغة التنفيذ
2026 م	العام الجامعي

تنفيذ الخوارزميات باستخدام Array و Queue و Priority Queue و Linked List و Stack

1. الفكرة العامة للكود

يحاكي الكود وجود خمس عمليات تتنافس على وحدة معالجة مركزية واحدة. لكل عملية أربعة بيانات أساسية: اسم العملية، ووقت وصولها، وزمن تنفيذها، وأولويتها. توجد هذه البيانات داخل الكود، ولذلك لا يطلب البرنامج أي إدخال عند التشغيل.

ينفذ البرنامج الخطوات الآتية تلقائيًا:

1. ينشئ العمليات الخمس من مصفوفة ثابتة.
2. يخزنها في قائمة مرتبطة.
3. يعرض بياناتها.
4. يشغل. FCFS.
5. يشغل. SJF.
6. يشغل. Priority Scheduling.
7. يشغل Round Robin بقيمة Quantum تساوي 2.
8. يحسب نتائج كل عملية.
9. يعرض مخطط Gantt لكل خوارزمية.
10. يعرض مقارنة نهائية بين الخوارزميات الأربع.

2. بيانات العينة المدمجة

العملية	Arrival Time	Burst Time	Priority
P1	2	5	3
P2	3	2	1
P3	4	8	4
P4	5	3	2
P5	7	4	1

قيمة الشريحة الزمنية:

Time Quantum = 2

تبدأ أول عملية عند الزمن 2 ، ولذلك يسجل البرنامج الفترة من 0 إلى 2 باسم IDLE.

3. ملفات الترويسة والمكتبات

يبدأ الملف بتضمين مكتبات C++ القياسية اللازمة. ويختلف دور كل مكتبة بحسب محتوى الكود، ومن أهم الاستخدامات:

المكتبة/النوع	الاستخدام
iostream	طباعة النتائج على الشاشة
iomanip	ترتيب الأعمدة وتحديد المنازل العشرية
string	حفظ أسماء العمليات والخوارزميات
algorithm	استخدام min و swap في التنفيذ والترتيب

المكتبة/النوع	الاستخدام
stdexcept	إطلاق استثناءات واضحة عند تجاوز السعة أو استخدام قيمة غير صحيحة
المصفوفات الخام []	حفظ العينة والنتائج والمؤشرات بأحجام ثابتة

استخدام:

```
using namespace std;
```

يسمح بكتابة `cout` بدل `std::cout` و `string` بدل `std::string`.

4. الثوابت وحدود التخزين

يحدد الكود حدين ثابتين عامين للسعات المستخدمة:

```
const int MAX_PROCESSES = 20;
const int MAX_SEGMENTS = 1000;
```

الأول هو الحد الأعلى لعدد العمليات، والثاني هو الحد الأعلى لمقاطع مخطط Gantt. كما توجد قيمة ثابتة محلية داخل الدالة `main` للشرية الزمنية المستخدمة في Round Robin:

```
const int timeQuantum = 2;
```

الفائدة من الثابت هي منع تغير القيمة أثناء التشغيل، وجعل موضع تعديلها واضحًا عند الحاجة إلى اختبار آخر.

5. بنية العملية Process

تمثل `Process` سجلًا واحدًا لعملية واحدة. وتحمل عادة الحقول الآتية:

الحقل	نوعه	وظيفته
pid	string	اسم العملية مثل P1
arrivalTime	عدد صحيح	وقت وصول العملية
burstTime	عدد صحيح	مدة التنفيذ المطلوبة
priority	عدد صحيح	رقم الأولوية؛ الأصغر أعلى

الفكرة ببساطة: بدل إنشاء متغيرات منفصلة لكل معلومة، يجمع `struct Process` بيانات العملية في كائن واحد.

6. بنية مقطع التنفيذ

يمثل كل مقطع في مخطط Gantt فترة زمنية واحدة. ويحتوي عادة على:

- اسم العملية أو كلمة `IDLE`.
- زمن بداية المقطع.
- زمن نهاية المقطع.

مثال:

```
[2-7: P1]
```

ويعني أن P1 استخدمت المعالج من الزمن 2 حتى الزمن 7.

7. بنية نتيجة الخوارزمية

يحتاج البرنامج إلى حفظ نتائج كل خوارزمية قبل عرض المقارنة. ولهذا تتضمن `ScheduleResult` العناصر الفعلية الآتية:

- اسم الخوارزمية.
- مصفوفة `metrics` التي تحفظ `Completion Time` و `Waiting Time` و `Turnaround Time` لكل عملية.
- مصفوفة `gantt` التي تحفظ مقاطع التنفيذ، ومتغير `segmentCount` لعددها.
- متوسط `Waiting Time`.
- متوسط `Turnaround Time`.

• CPU Busy Time.

• Total Simulation Time.

• CPU Utilization.

يُستنتج ترتيب التنفيذ من مصفوفة `Gantt`، ويُحسب `Idle Time` عند العرض من الفرق بين `Total Time` و `Busy Time`. وجود بنية نتيجة موحدة يجعل دالة العرض ودالة المقارنة تعملان مع الخوارزميات الأربع بالطريقة نفسها.

8. القائمة المرتبطة `ProcessLinkedList`.

8.1 وظيفتها

تستخدم الفئة `ProcessLinkedList` لتخزين العمليات الخمس في عقد مترابطة. تتكون كل عقدة من:

1. كائن `Process` يحمل بيانات العملية.
2. مؤشر `next` يشير إلى العقدة التالية.

وتحتفظ القائمة عادة بمؤشرين:

- `head`: أول عقدة.
- `tail`: آخر عقدة.

8.2 الإضافة إلى القائمة

عند إضافة عملية جديدة:

1. تُنشأ عقدة جديدة.
2. إذا كانت القائمة فارغة يصبح `head` و `tail` مساويين للعقدة الجديدة.
3. إذا لم تكن فارغة، يرتبط `tail->next` بالعقدة الجديدة.
4. يتحرك `tail` إلى آخر عقدة.

تستغرق الإضافة في النهاية زمنًا ثابتًا $O(1)$ عند وجود مؤشر `tail`.

8.3 نسخ القائمة إلى مصفوفة ثابتة

تحتاج الخوارزميات إلى الوصول إلى العمليات بواسطة الفهرس. لذلك توفر القائمة الدالة `copyToArray` التي تمر على العقد وتنسخ كل عملية إلى مصفوفة خام من النوع `Process destination[]`. يبقى الاستخدام الفعلي للقائمة واضحًا في التخزين، بينما تسهل المصفوفة الثابتة الحسابات والوصول المباشر إلى النتائج.

8.4 تحرير الذاكرة

تزيل دالة التنظيف العقد واحدة تلو الأخرى باستخدام `delete`. ويمنع ذلك تسرب الذاكرة بعد انتهاء البرنامج.

9. الطابور الدائري CircularQueue

9.1 الفكرة

يعمل CircularQueue وفق FIFO:

أول عنصر يدخل ← أول عنصر يخرج

ويستخدم مصفوفة داخلية ومؤشرات دائرية، بحيث يعود المؤشر إلى بداية المصفوفة عند الوصول إلى نهايتها.

9.2 المتغيرات الأساسية

تتضمن الفئة عادة:

- مصفوفة لحفظ فهارس العمليات.
- `frontIndex`: موضع أول عنصر.
- `rearIndex`: موضع الإدخال التالي أو آخر عنصر.
- `count`: عدد العناصر الحالية.

أما السعة فتحدد بثابت معروف في البرنامج، ولا توجد خانة مستقلة باسم `capacity` داخل الكائن.

9.3 عملية enqueue

تضيف `enqueue` فهرس العملية في نهاية الطابور، ثم تحرك المؤشر باستخدام باقي القسمة:

`rearIndex = (rearIndex + 1) % السعة_الثابتة_للمصفوفة`

إذا وصل `rearIndex` إلى آخر المصفوفة، يعود تلقائيًا إلى بدايتها.

9.4 عملية dequeue

تعيد `dequeue` العنصر الموجود في المقدمة، ثم تحرك `frontIndex` بالطريقة الدائرية نفسها.

9.5 سبب استخدامها

- في FCFS تحافظ على ترتيب وصول العمليات.
- في Round Robin تسمح بإعادة العملية غير المكتملة إلى نهاية الطابور بسهولة.

10. طابور الأولوية MinHeapPriorityQueue

10.1 الفكرة

تنفذ الفئة `MinHeapPriorityQueue` طابور أولوية باستخدام Min-Heap. العنصر الأفضل يوجد دائمًا عند الجذر، أي في الموضع صفر.

10.2 شكل الكومة

في المصفوفة التي تمثل الكومة، إذا كان موضع الأب `i` فإن:

الابن الأيسر = $i + 1 * 2$
الابن الأيمن = $i + 2 * 2$
الأب = $2 / (i - 1)$

10.3 الإدخال push

1. يضاف فهرس العملية إلى نهاية المصفوفة.
2. يقارن بالعنصر الأب.
3. إذا كان أفضل منه يتبادلان الموضع.
4. تتكرر المقارنة حتى يستقر ترتيب الكومة.

تسمى هذه العملية `heapify up`، وتعقيدها $O(\log n)$.

10.4 الاستخراج pop

1. يُحفظ عنصر الجذر لأنه المطلوب.
2. ينتقل آخر عنصر إلى الجذر.
3. يُحذف آخر موضع.
4. يُقارن الجذر بأبنائه ويتبادل مع الأفضل.
5. تستمر العملية حتى تستعيد الكومة ترتيبها.

تسمى هذه العملية `heapify down`، وتعقيدها $O(\log n)$.

10.5 معيار SJF

في SJF يكون العنصر الأفضل هو العملية ذات `burstTime` الأقل.
عند التعادل يكون الترتيب:

1. وقت الوصول الأقل.
2. ترتيب الإدخال الأقل.

10.6 معيار Priority Scheduling

في Priority يكون العنصر الأفضل هو العملية ذات رقم `priority` الأقل.
مثال:

Priority 1 أعلى من Priority 2

وعند التعادل يستخدم وقت الوصول، ثم ترتيب الإدخال.

11. المكس SegmentStack

11.1 وظيفته

يحفظ `SegmentStack` مقاطع التنفيذ وفق LIFO:

آخر مقطع يدخل ← أول مقطع يخرج

11.2 العمليات الأساسية

- `push`: إضافة مقطع إلى أعلى المكس.
 - `pop`: إزالة آخر مقطع مضاف.
 - `empty`: التحقق من خلو المكس.
- تُحفظ المقاطع في مصفوفة خام ثابتة داخل الفئة، ويحدد `topIndex` موضع أعلى عنصر؛ لذلك لا يحتاج المكس إلى إنشاء عقد ديناميكية.

11.3 دوره في المشروع

بعد بناء مخطط Gantt ، تُدفع المقاطع إلى المكس. وعند إخراجها يظهر سجل التنفيذ عكسيًا. هذا الاستخدام لا يغير نتيجة الجدولة، لكنه يطبق مفهوم Stack بصورة واضحة داخل المشروع.

12. ترتيب العمليات حسب الوصول

قبل بدء الجدولة، ينشئ البرنامج ترتيبًا لفهارس العمليات وفق:

1. Arrival Time الأقل.
 2. الموضع الأصلي الأقل في مصفوفة العمليات عند التعادل.
- الفائدة هي معرفة العملية التالية التي ستصل، وإضافة العمليات إلى Ready Queue في الوقت الصحيح.

13. الزمن الحالي وفترة الخمول

تعتمد الخوارزميات على متغير يمثل الزمن الحالي، مثل `currentTime`.
في بداية المثال:

```
currentTime = 0
أول Arrival Time = 2
```

وبما أنه لا توجد عملية جاهزة، يضيف البرنامج مقطعًا:

```
[0-2: IDLE]
```

ثم يجعل الزمن الحالي يساوي 2 ويبدأ تنفيذ P1.

14. شرح تنفيذ FCFS.

14.1 خطوات الخوارزمية في الكود

1. ترتيب العمليات حسب وقت الوصول.
2. إضافة العمليات الواصلة إلى `CircularQueue`.
3. إذا كان الطابور فارغًا، الانتقال إلى وقت وصول العملية التالية وتسجيل `IDLE`.
4. سحب عملية من مقدمة الطابور.
5. تنفيذها كاملة؛ لأن FCFS غير مستبقة.
6. تسجيل مقطع Gantt وزمن الإكمال.
7. إضافة أي عمليات وصلت أثناء التنفيذ.
8. تكرار الخطوات حتى اكتمال جميع العمليات.

14.2 التتبع على العينة

```
0-2   IDLE
2-7   P1
7-9   P2
9-17  P3
17-20 P4
20-24 P5
```

ترتيب التنفيذ:

P1 -> P2 -> P3 -> P4 -> P5

14.3 النتائج

WT = TAT-BT	TAT = CT-AT	CT	العملية
0	5	7	P1
4	6	9	P2
5	13	17	P3
12	15	20	P4
13	17	24	P5

Average WT = 6.80
Average TAT = 11.20

15. شرح تنفيذ SJF.

15.1 خطوات الخوارزمية في الكود

1. إضافة جميع العمليات التي وصلت إلى `MinHeapPriorityQueue`.
2. ضبط معيار الكومة ليعتمد Burst Time الأقل.
3. استخراج أقصر عملية جاهزة.
4. تنفيذها كاملة.
5. تحديث الزمن وإضافة العمليات التي وصلت أثناء التنفيذ.
6. تكرار الخطوات حتى اكتمال جميع العمليات.

15.2 لماذا تبدأ P1 ؟

عند الزمن 2 لا توجد إلا P1 ، ولذلك تبدأ حتى لو كانت هناك عملية أقصر ستصل لاحقًا. لا تنتظر SJF عملية لم تصل، كما أن النسخة المنفذة غير مستبقة فلا توقف P1 بعد بدء تنفيذها.

15.3 التتبع على العينة

بعد انتهاء P1 عند الزمن 7 ، تكون P2 و P3 و P4 و P5 قد وصلت. تُرتب حسب Burst Time:

P2 = 2
P4 = 3
P5 = 4
P3 = 8

مخطط التنفيذ:

0-2 IDLE
2-7 P1
7-9 P2
9-12 P4
12-16 P5
16-24 P3

ترتيب التنفيذ:

P1 -> P2 -> P4 -> P5 -> P3

15.4 النتائج

العملية	CT	TAT	WT
P1	7	5	0
P2	9	6	4
P3	24	20	12
P4	12	7	4
P5	16	9	5

Average WT = 5.00

Average TAT = 9.40

16. شرح تنفيذ Priority Scheduling.

16.1 خطوات الخوارزمية في الكود

1. إضافة العمليات الواصلة إلى MinHeapPriorityQueue.
2. ضبط المقارنة لتختار رقم Priority الأصغر.
3. استخراج العملية الأعلى أولوية.
4. تنفيذها كاملة لأن الخوارزمية غير مستبقة.
5. إضافة العمليات التي وصلت أثناء التنفيذ.
6. الاستمرار حتى اكتمال الجميع.

16.2 التتبع على العينة

تبدأ P1 وحدها عند الزمن 2 ، وتنتهي عند 7. بعد ذلك توجد عمليتان بأولوية 1 هما P2 و P5. تُختار P2 لأنها وصلت أولاً.

```
0-2  IDLE
2-7  P1
7-9  P2
9-13 P5
13-16 P4
16-24 P3
```

ترتيب التنفيذ:

P1 -> P2 -> P5 -> P4 -> P3

16.3 النتائج

WT	TAT	CT	العملية
0	5	7	P1
4	6	9	P2
12	20	24	P3
8	11	16	P4
2	6	13	P5

Average WT = 5.20
Average TAT = 9.60

17. شرح تنفيذ Round Robin.

17.1 المتغير الإضافي remainingTime

لأن العملية قد تُنفذ على عدة مرات، يحتاج البرنامج إلى مصفوفة تحفظ الزمن المتبقي لكل عملية.
في البداية:

```
remainingTime[i] = burstTime[i]
```

وبعد كل شريحة:

```
remainingTime = remainingTime - executedTime
```

17.2 خطوات الخوارزمية في الكود

1. إضافة العمليات الواصلة إلى CircularQueue.
2. إخراج أول عملية.
3. تنفيذ الأقل بين الزمن المتبقي و Time Quantum.
4. تحريك الزمن الحالي.
5. إضافة العمليات الجديدة التي وصلت أثناء الشريحة.
6. إذا بقي للعملية زمن، تُعاد إلى نهاية الطابور.
7. إذا انتهت، يسجل Completion Time.
8. تكرار الخطوات حتى اكتمال الجميع.

17.3 سبب إضافة الواصلين قبل إعادة العملية الحالية

إذا وصلت عمليات جديدة أثناء شريحة العملية الحالية، فقد كانت تنتظر بالفعل عند نهاية الشريحة. لذلك تدخل الطابور أولاً، ثم تُعاد العملية الحالية غير المكتملة إلى النهاية. تحقق هذه القاعدة ترتيبًا زمنيًا عادلاً.

17.4 التتبع على العينة

```
0-2  IDLE
2-4  P1
4-6  P2
```

```

6-8   P3
8-10  P1
10-12 P4
12-14 P5
14-16 P3
16-17 P1
17-18 P4
18-20 P5
20-22 P3
22-24 P3

```

ترتيب المقاطع:

```

P1 -> P2 -> P3 -> P1 -> P4 -> P5 -> P3 -> P1 -> P4 -> P5 -> P3 -> P3

```

17.5 النتائج

WT	TAT	CT	العملية
10	15	17	P1
1	3	6	P2
12	20	24	P3
10	13	18	P4
9	13	20	P5

Average WT = 8.40
Average TAT = 12.80

18. حساب المقاييس

بعد انتهاء الجدولة، تمر دالة الحساب على كل عملية وتنفذ:

```

TAT = CT - AT
WT  = TAT - BT

```

ثم تجمع القيم وتقسمها على عدد العمليات:

Average WT = Sum of WT / Number of Processes
Average TAT = Sum of TAT / Number of Processes

يستخدم البرنامج نوعًا عشريًا عند حساب المتوسط حتى لا يفقد الجزء العشري.

19. حساب CPU Utilization

مجموع Burst Time للعمليات الخمس:

$$5 + 2 + 8 + 3 + 4 = 22$$

إذن:

CPU Busy Time = 22

تبدأ المحاكاة عند الزمن 0 وتنتهي عند الزمن: 24

Total Simulation Time = 24

الفترة من 0 إلى 2 خمول:

CPU Idle Time = 2

الحساب:

$$\text{CPU Utilization} = (22 / 24) \times 100 = 91.67\%$$

تظهر النسبة نفسها في الخوارزميات الأربع؛ لأن الحمل واحد، ولا تُحتسب كلفة التبديل السياقي، ولا توجد خوارزمية تترك المعالج خاملًا مع وجود عملية جاهزة.

20. إضافة مقاطع Gantt.

تضيف دالة `addSegment` كل فترة تنفيذ بوصفها مقطعًا مستقلًا يحتوي على اسم العملية وبداية المقطع ونهايته. ولا تدمج الدالة المقاطع المتجاورة حتى إذا كانت للعملية نفسها؛ لأن إبقاء كل شريحة مستقلة يوضح عدد المرات التي حصلت فيها العملية على المعالج، وهو أمر مهم في Round Robin. ولهذا يظهر تنفيذ P3 من الزمن 20 إلى 22، ثم من 22 إلى 24، في مقطعين منفصلين؛ إذ يمثل كل مقطع شريحة زمنية مستقلة.

21. عرض النتائج.

تستخدم دوال العرض `setw` لترتيب الأعمدة و `fixed` مع `setprecision(2)` لعرض المتوسطات والنسب بمنزلتين عشريتين. يعرض كل قسم:

1. اسم الخوارزمية.
2. ترتيب التنفيذ.
3. مخطط Gantt.
4. جدول العمليات.
5. متوسط WT.
6. متوسط TAT.
7. Busy و Idle و Total Time.

8. CPU Utilization.

9. Reverse Execution Log الناتج من `SegmentStack`.

22. دالة المقارنة النهائية.

تستقبل دالة المقارنة نتائج الخوارزميات الأربع، ثم تطبع جدولًا موحدًا:

CPU Utilization	Average TAT	Average WT	الخوارزمية
91.67%	11.20	6.80	FCFS
91.67%	9.40	5.00	SJF

CPU Utilization	Average TAT	Average WT	الخوارزمية
91.67%	9.60	5.20	Priority
91.67%	12.80	8.40	Round Robin

وبحسب هذه العينة، تحقق SJF أقل متوسط انتظار وأقل متوسط دوران. ولا يعني ذلك أنها الأفضل دائمًا؛ فالنتيجة تتغير عند تغيير العمليات وأوقات وصولها.

23. شرح دالة main

تمثل `main` نقطة بداية البرنامج. ودورها مختصر وواضح:

1. إنشاء مصفوفة العينة المدمجة.
2. إنشاء `ProcessLinkedList`.
3. إضافة العمليات إلى القائمة.
4. تحويل البيانات إلى شكل مناسب للحساب.
5. عرض جدول العمليات.
6. استدعاء الخوارزميات الأربع.
7. حفظ النتائج.
8. عرض كل نتيجة.
9. استدعاء المقارنة النهائية.
10. إنهاء البرنامج بالقيمة صفر.

لا تحتوي `main` على `cin`؛ لأن التشغيل تلقائي ولا توجد مدخلات من المستخدم.

24. التعقيد الزمني

تنشئ الدالة `buildArrivalOrder` ترتيب الوصول باستخدام Selection Sort. ولهذا تكون مرحلة ترتيب العمليات $O(n^2)$. وبعد إنشاء ترتيب الوصول تكون تكلفة مرحلة الجدولة نفسها كما يأتي:

الجزء	التعقيد	السبب المختصر
<code>buildArrivalOrder</code>	$O(n^2)$	البحث عن العملية التالية داخل الجزء غير المرتب
FCFS بعد الترتيب	$O(n)$	معالجة كل عملية مرة واحدة
SJF بعد الترتيب	$O(n \log n)$	عمليات <code>push</code> و <code>pop</code> في Min-Heap
Priority بعد الترتيب	$O(n \log n)$	عمليات <code>push</code> و <code>pop</code> في Min-Heap
Round Robin بعد الترتيب	$O(d)$	<code>d</code> هو عدد شرائح التنفيذ المسجلة

وعند احتساب الترتيب مع الخوارزمية يضاف حد $O(n^2)$ إلى تكلفة الجدولة. أما المساحة المستخدمة فتتناسب خطيًا مع عدد العمليات ومقاطع التنفيذ ضمن الحدود الثابتة المعرفة في البرنامج.

25. كيف تبقى نتائج الخوارزميات مستقلة؟

تمرر الدالة `main` مصفوفة العمليات نفسها إلى الخوارزميات بصفة `const`، ولذلك لا تستطيع أي خوارزمية تغيير بيانات `arrivalTime` أو `burstTime` أو `priority` الأصلية. تنشئ كل خوارزمية كائن `ScheduleResult` مستقلًا، بينما تنشئ Round Robin مصفوفة محلية باسم `remainingTime` بدل تعديل `burstTime`. بهذه الطريقة تُقارن الخوارزميات الأربع على بيانات أصلية واحدة من دون اختلاط نتائجها.

26. قواعد مهمة تمنع الأخطاء.

1. لا تُضاف العملية إلى Ready Queue قبل Arrival Time.
2. لا يبدأ التنفيذ قبل الزمن الحالي.
3. لا تُسجل عملية مكتملة مرتين.
4. لا يكون Burst Time أو Quantum مساويًا للصفر.
5. يُسجل CT عند نهاية آخر تنفيذ للعملية.
6. تُحسب WT بعد معرفة CT و TAT.
7. يُستخدم فهرس العملية للربط بين الطوابير ومصفوفة البيانات.
8. تُحذف عقد `ProcessLinkedList` عند انتهاء عمر الكائن، بينما تستخدم بقية الهياكل مصفوفات ثابتة.
9. تعني Priority الأصغر أولوية أعلى في جميع أجزاء الكود.
10. تُستخدم قاعدة تعادل واحدة بصورة ثابتة.

27. الفرق بين الخوارزميات في سطر واحد

الخوارزمية	السؤال الذي تجيب عنه عند اختيار العملية التالية
FCFS	من وصل أولاً؟
SJF	من لديه أقل Burst Time بين الواصلين؟
Priority	من لديه أعلى أولوية بين الواصلين؟
Round Robin	من جاء دوره في Queue ؟

28. الفرق بين هياكل البيانات في سطر واحد

الهيكـل	القاعدة المختصرة
Array	عناصر متتابعة والوصول بالفهرس
Queue	FIFO
Priority Queue	الأفضل حسب مفتاح المقارنة يخرج أولاً
Linked List	عقد مرتبطة بمؤشرات
Stack	LIFO

29. أسئلة المناقشة المتوقعة وإجاباتها

السؤال 1: ما الهدف من المشروع؟

محاكاة أربع خوارزميات لجدولة المعالج باستخدام هياكل بيانات مناسبة، ثم مقارنتها بواسطة WT و TAT و CPU Utilization.

السؤال 2: لماذا لا يطلب البرنامج إدخالاً؟

لأن بيانات العمليات وقيمة Time Quantum معرفة داخل الكود، فيبدأ التشغيل تلقائيًا وبضمن استخدام العينة نفسها في كل مرة.

السؤال 3: ما الفرق بين Arrival Time و Burst Time ؟

Arrival Time هو وقت وصول العملية إلى النظام، أما Burst Time فهو الزمن الذي تحتاجه على المعالج.

السؤال 4: ما معنى Completion Time ؟

هو الزمن الذي تنتهي عنده العملية تمامًا.

السؤال 5: كيف يُحسب Turnaround Time ؟

$$TAT = CT - AT$$

السؤال 6: كيف يُحسب Waiting Time ؟

$$WT = TAT - BT$$

السؤال 7: لماذا استخدمت Queue في FCFS ؟

لأن FCFS تعتمد FIFO ؛ فالعملية التي تصل أولاً تنفذ أولاً.

السؤال 8: لماذا استخدمت Queue في Round Robin ؟

لأن العملية غير المكتملة تعاد إلى نهاية الطابور، ثم تحصل العملية الموجودة في المقدمة على الدور التالي.

السؤال 9: لماذا استخدمت Priority Queue في SJF ؟

لأنها تسمح باستخراج العملية ذات Burst Time الأقل من العمليات الجاهزة بكفاءة.

السؤال 10: لماذا استخدمت Priority Queue في Priority Scheduling ؟

لأنها تسمح باستخراج العملية ذات الأولوية الأعلى وفق مفتاح المقارنة.

السؤال 11: ما قاعدة الأولوية في المشروع؟

الرقم الأصغر يعني أولوية أعلى؛ فالأولوية 1 أعلى من الأولوية 2.

السؤال 12: ماذا يحدث عند تساوي الأولوية؟

تُختار العملية ذات Arrival Time الأقل، ثم ذات الموضع الأصلي الأقل في مصفوفة العمليات إذا استمر التعادل.

السؤال 13: هل SJF المنفذة مستبقة؟

لا، هي Non-Preemptive ؛ فالعملية تستمر حتى تنتهي بعد اختيارها.

السؤال 14: أي خوارزمية في المشروع مستبقة؟

Round Robin ، لأنها قد توقف العملية عند انتهاء Time Quantum قبل اكتمالها.

السؤال 15: ما قيمة Time Quantum ؟

وحدتان زمنيتان.

السؤال 16: ماذا لو كان Remaining Time أقل من Quantum ؟

تنفذ العملية للزمن المتبقي فقط، ثم تسجل مكتملة ولا تعاد إلى الطابور.

السؤال 17: ماذا لو كان Remaining Time أكبر من Quantum ؟

تنفذ لمدة Quantum ، ثم يطرح الزمن المنفذ وتُعاد إلى نهاية Queue.

السؤال 18: لماذا توجد فترة IDLE من 0 إلى 2 ؟

لأن أول عملية تصل عند الزمن 2 ، فلا توجد عملية جاهزة قبل ذلك.

السؤال 19: كيف حُسب CPU Busy Time ؟

بجمع Burst Time لجميع العمليات. $5 + 2 + 8 + 3 + 4 = 22$:

السؤال 20: كيف حُسب CPU Utilization ؟

بقسمة Busy Time على Total Simulation Time وضرب الناتج في 100 : $22 \div 24 \times 100 = 91.67\%$.

السؤال 21: لماذا تساوت CPU Utilization في الخوارزميات الأربع؟

لأن جميعها تنفذ الحمل نفسه، ولا تُحتسب كلفة Context Switch ، ولا تترك المعالج خاملاً عند وجود عملية جاهزة.

السؤال 22: ما وظيفة Linked List ؟

حفظ بيانات العمليات في عقد مترابطة وتطبيق مفهوم المؤشرات والإضافة إلى نهاية القائمة.

السؤال 23: ما وظيفة Stack ؟

حفظ مقاطع التنفيذ وعرضها بترتيب عكسي وفق LIFO.

السؤال 24: ما وظيفة Array ؟

حفظ العينة الثابتة، واستخدام الفهارس للوصول إلى العمليات والنتائج بسرعة.

السؤال 25: ما الفرق بين Queue و Stack ؟

Queue تعمل FIFO ، أما Stack فتعمل LIFO.

السؤال 26: ما الفرق بين Queue و Priority Queue ؟

Queue تخرج أقدم عنصر، أما Priority Queue فتخرج أفضل عنصر حسب معيار الأولوية.

السؤال 27: ما هو Min-Heap ؟

شجرة ثنائية مخزنة في مصفوفة يكون أفضل أو أصغر عنصر فيها عند الجذر.

السؤال 28: ما تعقيد push و pop في Min-Heap ؟

كل منهما $O(\log n)$.

السؤال 29: لماذا تُستخدم فهارس العمليات داخل الطوابير؟

لأن الفهرس صغير ويسمح بالوصول المباشر إلى بيانات العملية الأصلية داخل المصفوفة.

السؤال 30: أي خوارزمية أعطت أفضل نتيجة في العينة؟

SJF حققت أقل Average WT بقيمة 5.00 وأقل Average TAT بقيمة 9.40.

السؤال 31: هل SJF الأفضل دائماً؟

لا. تتغير النتيجة حسب بيانات العمليات، وقد تسبب SJF تأخر العمليات الطويلة.

السؤال 32: ما عيب FCFS ؟

قد تجعل عملية طويلة العمليات القصيرة تنتظر خلفها، وهو Convoy Effect.

السؤال 33: ما عيب Priority Scheduling ؟

قد تتأخر العمليات منخفضة الأولوية مدة طويلة، وتسمى الحالة Starvation.

السؤال 34: كيف يعالج النظام الحقيقي Starvation ؟

باستخدام Aging ، أي رفع أولوية العملية تدريجيًا كلما طال انتظارها.

السؤال 35: ما عيب Time Quantum الصغيرة جدًا؟

تزيد عدد Context Switches في النظام الحقيقي، فتزيد الكلفة التشغيلية.

السؤال 36: ماذا يحدث إذا كانت Quantum كبيرة جدًا؟

تصبح Round Robin قريبة من FCFS.

السؤال 37: لماذا لم تُحتسب Context Switch ؟

لتبسيط المحاكاة والتركيز على خوارزميات الجدولة وهياكل البيانات.

السؤال 38: كيف يتحقق الكود من عدم بدء العملية قبل وصولها؟

لا يضيفها إلى قائمة الجاهزين إلا إذا كان Arrival Time أقل من الزمن الحالي أو مساويًا له.

السؤال 39: لماذا توجد نسخة نتائج منفصلة لكل خوارزمية؟

لمنع اختلاط القيم المعدلة، ولضمان مقارنة الخوارزميات على البيانات الأصلية نفسها.

السؤال 40: لماذا نُشر المشروع على GitHub ؟

لحفظ الكود والتوثيق في مستودع منظم، وتسجيل الإصدارات، وتوفير رابط موحد للمراجعة.

30. ملخص حفظ سريع

```
FCFS    = First Arrival    = Queue
SJF     = Shortest Burst   = Priority Queue
Priority = Smallest PR Number = Priority Queue
RR      = Fixed Quantum    = Circular Queue
```

$TAT = CT - AT$

$WT = TAT - BT$

$CPU\ Utilization = Busy / Total \times 100$

Array = built-in data and indexed results

Linked List = process storage

Stack = reverse execution log

31. نتائج العينة للحفظ

CPU Utilization	Average TAT	Average WT	ترتيب التنفيذ الأساسي	الخوارزمية
91.67%	11.20	6.80	P1, P2, P3, P4, P5	FCFS
91.67%	9.40	5.00	P1, P2, P4, P5, P3	SJF
91.67%	9.60	5.20	P1, P2, P5, P4, P3	Priority
91.67%	12.80	8.40	P1, P2, P3, P1, P4, P5, P3, P1, P4, P5, P3, P3	Round Robin